

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Database Management Systems

**COURSE CODE:** CSA05

**COURSE OUTCOME COVERED**

**CO5:** *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool: Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 40%

### QUESTIONS

| Q.No | Question                                                                                                                                                             | Bloom's Level |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1    | Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.                      | BL4 - Analyze |
| 2    | Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes. | BL4 - Analyze |
| 3    | Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.                                      | BL3 - Apply   |
| 4    | Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.                                            | BL4 - Analyze |
| 5    | Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.                                         | BL3 - Apply   |
| 6    | Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.                                       | BL4 - Analyze |
| 7    | Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.                                | BL3 - Apply   |
| 8    | Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.                                             | BL3 - Apply   |
| 9    | Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.                           | BL4 - Analyze |
| 10   | Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.                       | BL4 - Analyze |

## ANSWERS

### Q1.

Sample query used for this answer:

```
SELECT c.name, o.order_date
FROM Customers c, Orders o
WHERE c.cid = o.cid
      AND c.city = 'Chennai'
      AND o.amount > (SELECT AVG(o2.amount)
                      FROM Orders o2
                      WHERE o2.cid = c.cid)
```

Initial (unoptimized) execution plan, as produced directly from the parsed query:

```
      Project (name, order_date)
        |
      Select (c.city='Chennai' AND c.cid=o.cid
              AND o.amount > AVG_subquery)
        |
    Cartesian Product
      /      \
Customers c   Orders o

(correlated subquery AVG(o2.amount) WHERE o2.cid=c.cid
evaluated once per row produced above)
```

#### Heuristic optimization, applied step-by-step:

Step 1 - Split the conjunctive selection into individual selections so each predicate can be moved independently:  $c.city='Chennai'$ ,  $c.cid=o.cid$ , and  $o.amount > AVG(...)$  become three separate Select nodes.

Step 2 - Push selections down as close to their base relations as possible.  $c.city='Chennai'$  involves only Customers, so it moves directly below the Customers scan. The join predicate  $c.cid=o.cid$  stays at the point where both relations meet, and the correlated amount filter stays with Orders since it references  $o.amount$  and the outer  $c.cid$ .

Step 3 - Replace the Cartesian Product followed by the join-predicate Select with a single Join operator (Customers JOIN Orders ON  $c.cid=o.cid$ ), which is always cheaper than materializing the full cross product first.

Step 4 - Push projections down: only  $c.name$ ,  $c.cid$ ,  $o.order\_date$ ,  $o.amount$ ,  $o.cid$  are ever needed, so a Project is inserted directly above each base-table scan to shrink every intermediate result as early as possible.

Step 5 - For the correlated subquery, push the  $AVG(amount)$  computation down so it is grouped and computed once per cid via a Group-By on Orders, rather than being recomputed per outer row where possible.

Resulting optimized plan:

```
      Project (name, order_date)
        |
      Select (o.amount > avg_amt)
        |
      Join (c.cid = o.cid)
        /      \
Select(city='Chennai')  Join (o2.cid = c.cid)
  |                      /      \
Project(cid,name)      Project(cid,  GroupBy cid,
  |                      amount)      AVG(amount)
Customers c            Orders o      Orders o2
```

### Q2.

Given relations: R has  $n_R = 10,000$  tuples stored in  $b_R = 100$  pages; S has  $n_S = 50,000$  tuples stored in  $b_S = 500$  pages. Available buffer:  $B = 22$  pages.

**Block Nested-Loop Join (R as the smaller, outer relation):**

Cost =  $b_R + \text{ceil}(b_R / (B-2)) \times b_S = 100 + \text{ceil}(100/20) \times 500 = 100 + 5 \times 500 = 2,600 + 100 = 2,700$  I/Os.

**Sort-Merge Join:**

Assuming external merge sort is needed for both relations followed by a single merge pass: Cost  $\sim 3 \times (b_R + b_S) = 3 \times 600 = 1,800$  I/Os.

**Hash Join (Grace Hash Join, build phase on the smaller relation R):**

Since  $B$  (22) is close to  $\text{sqrt}(b_S) = \text{sqrt}(500) \sim 22.4$ , a single partitioning pass is (just) sufficient. Cost  $\sim 3 \times (b_R + b_S) = 3 \times 600 = 1,800$  I/Os.

**Comparison and decision:**

| Join Algorithm         | Estimated Cost (I/Os) |
|------------------------|-----------------------|
| Block Nested-Loop Join | 2,700                 |
| Sort-Merge Join        | 1,800                 |
| Hash Join              | 1,800                 |

Block Nested-Loop Join is ruled out as the most expensive option. Between Sort-Merge and Hash Join, both have equal estimated I/O cost here, so the optimizer breaks the tie using secondary factors: if the query (or a later operator, such as ORDER BY or GROUP BY) needs the output sorted on the join key, Sort-Merge Join is chosen since it produces sorted output as a side effect; otherwise Hash Join is preferred because it avoids the CPU overhead of sorting and performs well as long as one relation's partitions fit in memory.

### Q3.

JDBC connectivity to MySQL using PreparedStatement, with connection handling and exception-safe cleanup:

```
import java.sql.*;

public class OrderRepository {

    private static final String URL =
        "jdbc:mysql://localhost:3306/company_db";
    private static final String USER = "app_user";
    private static final String PASSWORD = "app_pass";

    public static void insertOrder(int customerId, double amount, String orderDate) {
        String sql = "INSERT INTO Orders (cid, amount, order_date) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, customerId);
            stmt.setDouble(2, amount);
            stmt.setString(3, orderDate);

            int rowsInserted = stmt.executeUpdate();
            System.out.println(rowsInserted + " row(s) inserted.");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
}

public static void getOrdersAboveAmount(int customerId, double minAmount) {
    String sql = "SELECT order_date, amount FROM Orders WHERE cid = ? AND amount > ?";

    try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, customerId);
        stmt.setDouble(2, minAmount);

        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                System.out.println(rs.getString("order_date") + " - " +
rs.getDouble("amount"));
            }
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

#### Q4.

Given schedule S over four transactions T1, T2, T3, T4:

| Step | Transaction | Operation |
|------|-------------|-----------|
| 1    | T1          | Read(A)   |
| 2    | T2          | Read(A)   |
| 3    | T1          | Write(A)  |
| 4    | T3          | Read(B)   |
| 5    | T2          | Write(B)  |
| 6    | T4          | Read(C)   |
| 7    | T3          | Write(C)  |
| 8    | T4          | Write(A)  |

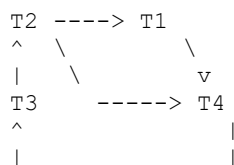
**Conflicting operation pairs (same data item, at least one Write, in schedule order):**

On A: T2 Read(A) before T1 Write(A) -> edge T2 -> T1. T1 Write(A) before T4 Write(A) -> edge T1 -> T4. T2 Read(A) before T4 Write(A) -> edge T2 -> T4.

On B: T3 Read(B) before T2 Write(B) -> edge T3 -> T2.

On C: T4 Read(C) before T3 Write(C) -> edge T4 -> T3.

**Precedence graph:**



Edges:  $T2 \rightarrow T1$ ,  $T1 \rightarrow T4$ ,  $T2 \rightarrow T4$ ,  $T3 \rightarrow T2$ ,  $T4 \rightarrow T3$

Cycle check:  $T3 \rightarrow T2 \rightarrow T1 \rightarrow T4 \rightarrow T3$  is a directed cycle (every edge in it,  $T3 \rightarrow T2$ ,  $T2 \rightarrow T1$ ,  $T1 \rightarrow T4$ ,  $T4 \rightarrow T3$ , is present in the graph).

**Conclusion:** since the precedence graph contains a cycle, schedule S is NOT conflict-serializable. No equivalent serial order of T1, T2, T3, T4 exists that would produce the same result as this interleaving; the DBMS's concurrency control protocol must prevent this exact interleaving from being allowed to occur.

## Q5.

Transaction set: T1 needs locks on A then B; T2 needs locks on B then A.

| Time | Action                | Result                         |
|------|-----------------------|--------------------------------|
| t1   | T1 requests X-lock(A) | Granted                        |
| t2   | T2 requests X-lock(B) | Granted                        |
| t3   | T1 requests X-lock(B) | Denied - held by T2 - T1 waits |
| t4   | T2 requests X-lock(A) | Denied - held by T1 - T2 waits |

At t4 both transactions are blocked waiting on each other. The wait-for graph has edge  $T1 \rightarrow T2$  (T1 waits for T2's lock on B) and edge  $T2 \rightarrow T1$  (T2 waits for T1's lock on A), which is a 2-node cycle: a deadlock.

Resolution: the deadlock detector picks a victim to abort, typically the transaction that has done the least work so far (or the youngest). Aborting T2 releases its X-lock(B); the lock manager grants T1's pending request for B; T1 completes and commits, releasing both A and B; T2 is then restarted from the beginning and completes successfully on its retry.

## Q6.

Transactions with timestamps:  $TS(T1) = 5$  (older),  $TS(T2) = 10$  (younger).

**Scenario A - T1 (older) requests a lock currently held by T2 (younger):**

Wait-Die: the requester (T1) is older than the holder (T2), so the older transaction is allowed to wait. T1 waits for T2 to release the lock.

Wound-Wait: the requester (T1) is older than the holder (T2), so T1 "wounds" T2 - T2 is forced to abort and restart (with the same timestamp), and T1 acquires the lock once T2 releases it.

**Scenario B - T2 (younger) requests a lock currently held by T1 (older):**

Wait-Die: the requester (T2) is younger than the holder (T1), so T2 "dies" - it aborts immediately and restarts later with the same original timestamp (so it eventually becomes the oldest and wins).

Wound-Wait: the requester (T2) is younger than the holder (T1), so T2 simply waits for T1 to release the lock; no abort happens in this case.

Both schemes always resolve conflicts using timestamp order alone, so a transaction can only ever wait for an older transaction (Wait-Die) or be waited-for by a younger one (Wound-Wait) - this monotonic ordering makes a circular wait, and therefore a deadlock, impossible under either scheme.

## Q7.

Given transaction log (Immediate Update policy, values shown as old,new):

```
<T1 start>
<T1, A, 10, 20>
<T2 start>
<T2, B, 5, 15>
<CHECKPOINT {T1, T2}>
<T1, C, 7, 17>
<T1 commit>
<T3 start>
<T3, D, 1, 9>
----- SYSTEM CRASH -----
```

#### Applying the Immediate Update recovery algorithm:

Analysis pass: scan the log from the last checkpoint forward. T1 has a commit record -> REDO list. T2 and T3 have no commit or abort record before the crash -> UNDO list.

Undo phase (scan backward, restore old values, for T2 and T3 only): T3, D reverts to 1. T2, B reverts to 5.

Redo phase (scan forward, reapply new values, for T1 only): T1, A is set to 20. T1, C is set to 17.

**Final recovered state: A=20, B=5, C=17, D=1. T1's committed changes are fully durable; T2 and T3, which never committed, have no trace of their partial updates left in the database.**

## Q8.

Given log file (Deferred Update policy - writes are buffered and only applied to disk after commit):

```
<T1 start>
<T1, X, new=100>
<T1 commit>
<T2 start>
<T2, Y, new=200>
----- SYSTEM CRASH (T2 not committed) -----
```

#### Applying the Deferred Update (NO-UNDO/REDO) recovery algorithm:

Because updates are only ever written to disk after commit, no uncommitted transaction can have modified the disk copy of the database - so there is nothing to undo, regardless of how far T2 had progressed.

Analysis pass: T1 has a commit record -> add to REDO list. T2 has no commit record at the time of the crash -> its logged update is simply discarded (an <ABORT T2> record is written); no undo action is needed since T2's write never reached disk.

Redo phase: reapply T1's committed update - X is set to 100.

**Final recovered state: X=100; Y is unaffected by T2's incomplete work, since it was never written to disk in the first place.**

## Q9.

Banking scenario: T1 transfers funds from account A to account B; T2 concurrently reads the balance of A. Concurrency control mechanism: Strict Two-Phase Locking.

| Time | T1 (transfer A -> B)         | T2 (read balance of A) |
|------|------------------------------|------------------------|
| t1   | Requests X-lock(A) - granted |                        |
| t2   | Reads A, deducts amount      |                        |

| Time | T1 (transfer A -> B)                        | T2 (read balance of A)                                        |
|------|---------------------------------------------|---------------------------------------------------------------|
| t3   |                                             | Requests S-lock(A) - BLOCKED<br>(T1 holds X-lock)             |
| t4   | Requests X-lock(B) - granted                |                                                               |
| t5   | Writes new balance of A                     |                                                               |
| t6   | Credits B, writes new balance of B          |                                                               |
| t7   | COMMITTS - releases X-lock(A) and X-lock(B) |                                                               |
| t8   |                                             | S-lock(A) granted - reads the final,<br>post-transfer balance |

Under strict 2PL, T1 holds its exclusive locks on A and B for the entire transaction, right up until it commits (not just until it finishes using them). This means T2's request to read A is blocked at t3 and cannot proceed until T1 fully commits at t7 - so T2 can never observe the intermediate, half-completed state of the transfer (debited from A but not yet credited to B). By the time T2's read is granted at t8, it sees the fully consistent, post-commit balance. This is exactly the guarantee strict 2PL is designed to provide: it eliminates dirty reads and, because locks are held until commit, also prevents cascading rollbacks if T1 were to abort instead.

## Q10.

ALGORITHM HeuristicQueryOptimizer(queryTree)

INPUT : queryTree - initial logical plan (Select, Project, Join, Cartesian Product nodes)

OUTPUT: optimizedTree - heuristically rewritten plan

BEGIN

tree <- SplitConjunctiveSelections(queryTree)

FOR EACH selectNode IN tree DO

target <- LowestNodeThatStillHasAllAttributesOf(selectNode)

MoveSelectNode(selectNode, target)

END FOR

FOR EACH pair (cartesianNode, selectNode) IN tree

WHERE selectNode IS the join predicate ABOVE cartesianNode DO

ReplaceWithJoin(cartesianNode, selectNode)

END FOR

FOR EACH node IN tree DO

neededAttributes <- AttributesUsedAbove(node)

InsertProjectNode(node, neededAttributes)

END FOR

joinList <- AllJoinNodesIn(tree)

SORT joinList BY EstimatedIntermediateResultSize ASCENDING

tree <- ReorderJoins(tree, joinList)

optimizedTree <- tree

RETURN optimizedTree

END

## Code of Conduct:

*I Dheeraj Krushna/192525230 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

*P. DheerajKrushna*

## RUBRICS FOR CALCULATION:

| Criteria                | Weightage | Level 4 (Exemplary)                                                            | Level 3 (Proficient)                                  | Level 2 (Developing)                          | Level 1 (Beginning)                      |
|-------------------------|-----------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Problem Understanding   | 25%       | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |
| Constraint Analysis     | 25%       | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |
| Solution Development    | 25%       | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |
| Application of Concepts | 15%       | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |
| Justification           | 10%       | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |